VITRYNE

Marketplace de Moda Regional com Delivery

Diagrama de Classes

Artefato de Modelagem e Projeto — UML

Data: 06/09/2026

1. Sumário

1. Sumário 2
2. Identificação 4
3. Objetivo 4
4. Definições e Siglas 4
5. Visão Geral do Diagrama 6
6. Convenções e Notação UML Adotadas 6
6.1. Estrutura das Classes 6
6.2. Visibilidade 6
6.3. Relacionamentos 6
7. Classes por Domínio 8
7.1. Identidade e Autenticação 8
Usuario 8
Cliente (especialização de Usuario) 8
Lojista (especialização de Usuario) 9
Entregador (especialização de Usuario) 10
7.2. Endereçamento e Privacidade (LGPD) 10
Endereco 10
Consentimento (nova classe) 11
SolicitacaoExclusaoConta (nova classe — UC17) 12
7.3. Catálogo e Estoque 12
Loja 12
Produto 13
Estoque 14
Cupom (nova classe — UC27, UC49) 14
7.4. Carrinho de Compras 14
CarrinhoCompras 15
ItemCarrinho 15
Favorito (nova classe — UC24) 15
7.5. Pedidos 16
Pedido 16
ItemPedido 17
HistoricoStatusPedido (nova classe — UC30, UC58) 17
7.6. Pagamento e Financeiro 18
Pagamento 18
Cartao 18
ContaBancaria 19
TaxaComissao (nova classe — UC63) 19
Repasse (nova classe — UC47) 20
7.7. Logística e Entrega 20

Entrega **20**
ProblemaEntrega **21**
7.8. Suporte, Engajamento e Auditoria **21**
Notificacao **21**
Avaliacao **22**
TutorialConcluido (nova classe — UC64) 23
LogAuditoria (nova classe — UC13, UC48, UC61, UC62) 23
8. Relacionamentos Principais **24**
9. Considerações Finais **25**

2. Identificação

Projeto	Vitryne — Escola de TI
Organização	Vitryne
Equipe	Bruno Abrahim · Gabriel Rodrigues · Henrique Pacheco · Hugo Zuin · João Ehlers · Leonardo Xavier
Autores	João Ehlers · Henrique Pacheco
Data de Criação	27/05/2026
Última Revisão	06/09/2026

3. Objetivo

Este documento apresenta o Diagrama de Classes (UML) elaborado na etapa de Modelagem e Projeto da plataforma Vitryne. Destina-se à equipe de desenvolvimento e aos revisores acadêmicos, comunicando a estrutura orientada a objetos do sistema — classes, atributos, métodos e relacionamentos — que servirá de base para a implementação das entidades JPA, dos serviços de domínio e dos contratos de API. O artefato documenta cada classe individualmente com suas responsabilidades, formaliza a especialização de Usuario nos três perfis de ator e mapeia os relacionamentos com suas cardinalidades.

4. Definições e Siglas

Glossário dos termos técnicos e siglas utilizados neste documento, em ordem alfabética.

TERMO / SIGLA	DEFINIÇÃO
Agregação	Relacionamento em que uma classe contém outra, mas as instâncias têm ciclo de vida independente.
Associação	Relacionamento estrutural entre duas classes que indica que uma usa ou conhece a outra.
Atributo	Variável de estado pertencente a uma classe (precedida por sinal '-' indica privado).
Cardinalidade	Número de instâncias envolvidas em um relacionamento (1, 0..1, 0..*, 1..*, etc.).
Classe	Estrutura que define atributos e métodos de objetos do domínio.
Composição	Relacionamento em que a existência da parte depende do todo (ciclo de vida acoplado).
DTO	Data Transfer Object — objeto usado para transportar dados entre camadas.
Especialização	Relacionamento de herança entre uma superclasse e suas subclasses.
Generalização	Relacionamento UML que indica que uma classe é uma forma genérica de outras (ISA).
Método	Operação ou comportamento exposto por uma classe (precedido por sinal '+' indica público).

TERMO / SIGLA	DEFINIÇÃO
MVC	Model-View-Controller — padrão arquitetural que separa responsabilidades.
OO	Orientação a Objetos — paradigma baseado em classes e objetos.
POJO	Plain Old Java Object — objeto Java simples, sem dependências de frameworks.
UML	Unified Modeling Language — linguagem padrão de modelagem de software.
Visibilidade	Modificador de acesso: '+' público, '-' privado, '#' protegido.

Tabela 1 — Glossário de termos e siglas.

5. Visão Geral do Diagrama

Visão consolidada do Diagrama de Classes da plataforma Vitryne, com as 28 classes identificadas e seus respectivos relacionamentos. As subseções da Seção 7 detalham cada classe individualmente, com atributos e métodos.

1. **Identidade e Autenticação:** Usuario, Cliente, Lojista, Entregador
2. **Endereçamento e Privacidade (LGPD):** Endereco, Consentimento, SolicitacaoExclusaoConta
3. **Catálogo e Estoque:** Loja, Produto, Estoque, Cupom
4. **Carrinho de Compras:** CarrinhoCompras, ItemCarrinho, Favorito
5. **Pedidos:** Pedido, ItemPedido, HistoricoStatusPedido
6. **Pagamento e Financeiro:** Pagamento, Cartao, ContaBancaria, TaxaComissao, Repasse
7. **Logística e Entrega:** Entrega, ProblemaEntrega
8. **Suporte, Engajamento e Auditoria:** Notificacao, Avaliacao, TutorialConcluido, LogAuditoria

{INSERIR IMAGEM E LINK}

Figura 1 — Diagrama de Classes (UML) consolidado da plataforma Vitryne.

6. Convenções e Notação UML Adotadas

Para garantir consistência na leitura e na interpretação do modelo, foram adotadas as convenções descritas a seguir.

6.1. Estrutura das Classes

- Cada classe é representada como um retângulo dividido em três compartimentos: nome, atributos e métodos.
- Atributos seguem a notação UML: visibilidade + nome + tipo (ex.: -nome: String).
- Métodos seguem a notação UML: visibilidade + nome(parâmetros) + tipo de retorno (ex.: +login(email, senha): String).

6.2. Visibilidade

- Sinal '+' indica visibilidade pública (acessível externamente).
- Sinal '-' indica visibilidade privada (acesso interno à classe).
- Sinal '#' indica visibilidade protegida (acessível a subclasses).

6.3. Relacionamentos

- Associação: linha sólida entre duas classes, com nome textual (ex.: "possui", "gerencia").
- Generalização (herança): seta com triângulo vazado apontando para a superclasse (Usuario).
- Cardinalidades: representadas nas extremidades das linhas (1, 0..1, 0..*, 1..*, etc.).
- Os relacionamentos são detalhados na Seção 8 com cardinalidade e significado semântico.

7. Classes por Domínio

Esta seção descreve cada uma das 28 classes do modelo, organizadas em oito domínios funcionais, com as tabelas completas de atributos e métodos de cada uma.

7.1. Identidade e Autenticação

Usuario

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
usuario	String	Login.
email	String	E-mail único.
senhaHash	String	Hash BCrypt.
role	String	CONSUMER, STORE, DELIVERER, ADMIN.
status	String	ATIVO, SUSPENSO, DESATIVADO. *(novo)*
provedorAuth	String	LOCAL, GOOGLE. *(novo)*
provedorId	String	ID externo do provedor OAuth. *(novo)*
preferenciasNotificacao	JSON	Canais habilitados. *(novo)*
tentativasLoginFalhas	Integer	Contador para bloqueio. *(novo)*
bloqueadoAte	DateTime	Fim do bloqueio temporário. *(novo)*
criadoEm	DateTime	Data de criação.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
login(email, senha)	String	Autentica e retorna token JWT.
logout()	void	Invalida a sessão atual.
alterarSenha(senhaAtual, novaSenha)	void	Altera a senha mediante reautenticação.
recuperarSenha(email)	void	Inicia fluxo de recuperação.
vincularProvedorOAuth(provedor, providerId)	void	Vincula conta Google a cadastro existente. *(novo)*
registrarTentativaFalha()	void	Incrementa contador e aplica bloqueio ao atingir o limite. *(novo)*
desbloquear()	void	Remove bloqueio temporário expirado. *(novo)*

Cliente *(especialização de Usuario)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
nome	`String`	Nome completo.
cpf	`String`	CPF.
rg	`String`	RG.
telefone	`String`	Telefone.
dataNascimento	`Date`	Data de nascimento.
raioBuscaKm	`Integer`	Raio de busca configurado. *(novo)*

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
adicionarAoCarrinho(produto, tamanho, qtd)	`void`	Adiciona item ao carrinho.
realizarPedido()	`Pedido`	Confirma checkout e cria pedido.
solicitarDevolucao(pedido, motivo)	`void`	Solicita devolução de pedido entregue.
visualizarHistorico()	`List<Pedido>`	Lista pedidos do cliente.
adicionarFavorito(produto)	`void`	*(novo)*
removerFavorito(produto)	`void`	*(novo)*
definirRegiaoBusca(raioKm)	`void`	*(novo)*
solicitarExclusaoConta(motivo)	`void`	*(novo)*

Lojista *(especialização de Usuario)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
nome	`String`	Nome completo.
cnpj	`String`	CNPJ.
telefone	`String`	Telefone.
dataNascimento	`Date`	Data de nascimento.
statusAprovacao	`String`	PENDENTE_APROVACAO, APROVADO, REPROVADO. *(novo)*
motivoReprovacao	`String`	Preenchido quando reprovado. *(novo)*

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
confirmarPedido(pedidoId)	`void`	Confirma atendimento do pedido.
recusarPedido(pedidoId, motivo)	`void`	Recusa pedido informando motivo.
marcarProntoParaColeta(pedidoId)	`void`	*(novo — UC46)*
cadastrarProduto(produto)	`void`	Cadastra produto no catálogo.
visualizarVendas()	`List<Pedido>`	Histórico de vendas.

MÉTODO	RETORNO	DESCRIÇÃO
`configurarDisponibilidadeLoja(status)`	`void`	ABERTA, FECHADA, PAUSA_TEMPORARIA. *(novo)*
`criarCupom(cupom)`	`void`	*(novo — UC49)*
`atualizarDadosBancarios(conta)`	`void`	*(novo — UC48)*
`definirLimiarAvisoEstoque(qtd, aplicarATodos)`	`void`	*(novo — UC66)*

Entregador *(especialização de Usuario)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
`nome`	`String`	Nome completo.
`cpf`	`String`	CPF.
`cnh`	`String`	Número da CNH.
`placaVeiculo`	`String`	Placa do veículo.
`tipoVeiculo`	`String`	Moto, carro, bicicleta, etc.
`telefone`	`String`	Telefone.
`dataNascimento`	`Date`	Data de nascimento.
`disponivel`	`Boolean`	ONLINE para receber entregas.
`statusCadastro`	`String`	EM_ANALISE, APROVADO, REPROVADO. *(novo)*
`documentosUrls`	`List<String>`	CNH, CRLV, comprovante, selfie. *(novo)*

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
`aceitarEntrega(entregaId)`	`void`	Aceita entrega ofertada.
`recusarEntrega(entregaId)`	`void`	Recusa entrega ofertada.
`confirmarColeta(entregaId, codigo)`	`void`	Valida código de coleta (geofencing). *(alterado — recebe código)*
`confirmarEntrega(entregaId, codigo)`	`void`	Valida código de entrega (geofencing). *(alterado — recebe código)*
`alternarDisponibilidade()`	`void`	Alterna ONLINE/OFFLINE.
`registrarProblema(entregaId, tipo, descricao)`	`void`	*(novo — UC54)*

7.2. Endereçamento e Privacidade (LGPD)

Endereco

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	`Long`	Identificador único.
clienteId	`Long`	
cep	`String`	CEP.
logradouro	`String`	Logradouro.
numero	`String`	Número.
complemento	`String`	Complemento.
bairro	`String`	Bairro.
cidade	`String`	Cidade.
estado	`String`	Estado.
latitude	`Double`	Latitude geocodificada.
longitude	`Double`	Longitude geocodificada.
apelido	`String`	"Casa", "Trabalho". *(novo — UC14)*
padrao	`Boolean`	Endereço padrão do usuário.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
validarCep()	`Boolean`	Valida CEP via serviço externo.
geocodificar()	`void`	Obtém coordenadas geográficas.

Consentimento *(nova classe)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	`Long`	Identificador único.
usuarioId	`Long`	
tipo	`String`	TERMOS_USO, PRIVACIDADE, COOKIES_ANALYTICS, COOKIES_MARKETING.
versaoTermos	`String`	Versão aceita.
dataHoraAceite	`DateTime`	Momento do aceite.
revogadoEm	`DateTime`	Momento da revogação (nullable).

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
registrar()	`void`	Registra o aceite.
revogar()	`void`	Revoga o consentimento opcional.

SolicitacaoExclusaoConta *(nova classe — UC17)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
usuarioId	Long	
motivo	String	Motivo opcional.
status	String	SOLICITADA, CANCELADA, CONCLUIDA.
dataSolicitacao	DateTime	Data da solicitação.
prazoConclusao	DateTime	Até 15 dias úteis.
dataConclusao	DateTime	Nullable.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
solicitar()	void	Registra a solicitação.
cancelar()	void	Cancela dentro do prazo de carência.
concluir()	void	Efetiva a exclusão/anonimização.

7.3. Catálogo e Estoque

Loja

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
lojistaId	Long	
enderecoId	Long	Endereço físico. *(novo)*
nome	String	Nome fantasia.
cnpj	String	CNPJ.
descricao	String	Descrição pública.
logoUrl	String	URL do logotipo.
bannerUrl	String	URL do banner.
status	String	ATIVA, PENDENTE_APROVACAO, REPROVADA, FECHADA, PAUSADA, SUSPENSA, INATIVA. *(ampliado)*
horarioFuncionamento	String (JSON)	Por dia da semana.
raioEntregaKm	Integer	Raio máximo de entrega.
limiarAvisoEstoquePadrao	Integer	Limiar padrão da loja. *(novo — UC66)*

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
abrirLoja()	void	Abre manualmente.
fecharLoja()	void	Fecha manualmente.
pausarLoja()	void	Pausa temporária.
estaAberta()	Boolean	Status atual.
calcularDistancia(lat, lng)	Double	Distância até uma coordenada.
definirLimiarAvisoEstoque(qtd, aplicarATodosProdutos)	void	*(novo — UC66)*

Produto

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
lojaId	Long	Loja à qual pertence. *(novo)*
nome	String	Nome do produto.
descricao	String	Descrição.
preco	Double	Preço de venda.
precoPromocional	Double	Preço promocional ativo.
dataInicioPromocao	DateTime	*(novo — UC39)*
dataFimPromocao	DateTime	*(novo — UC39)*
tipo	String	Categoria.
cor	String	Cor.
avaliacao	Float	Média (0–5).
status	String	ATIVO, INATIVO.
limiarAvisoEstoque	Integer	Sobrepõe o limiar da loja. *(novo — UC66)*
fotosUrls	List<String>	URLs das fotos.
criadoEm	DateTime	Base do selo de novidade. *(novo)*

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
verificarDisponibilidade()	Boolean	Há estoque disponível (algum tamanho).
configurarPromocao(precoPromocional, dataInicio, dataFim)	void	*(substitui aplicarDesconto)*
calcularPrecoFinal()	Double	Considera promoções vigentes.
estaEmPromocao()	Boolean	*(novo)*
isNovidade()	Boolean	Cadastrado há ≤7 dias. *(novo)*

Estoque

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
produtoId	Long	Produto ao qual pertence.
tamanho	String	**Novo e crítico** (UC41).
quantidade	Integer	Quantidade disponível para o tamanho.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
diminuirEstoque(qtd)	void	Decrementa após venda.
aumentarEstoque(qtd)	void	Incrementa (reposição/devolução).
estaDisponivel()	Boolean	Ao menos 1 unidade.

Cupom *(nova classe — UC27, UC49)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
lojaId	Long	
codigo	String	Código do cupom.
tipoDesconto	String	PERCENTUAL, VALOR_FIXO.
valorDesconto	Double	Valor/percentual.
valorMinimoPedido	Double	Valor mínimo exigido.
limiteUso	Integer	Limite total de usos.
usosRealizados	Integer	Contador de usos.
dataInicio	DateTime	Início da vigência.
dataFim	DateTime	Fim da vigência.
status	String	ATIVO, ENCERRADO.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
validar(pedido)	Boolean	Verifica vigência, mínimo e aplicabilidade.
aplicar(pedido)	void	Aplica o desconto e incrementa uso.
encerrar()	void	Interrompe novas aplicações.

7.4. Carrinho de Compras

CarrinhoCompras

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
clienteId	Long	
previsaoValorTotal	Double	Valor total estimado.
atualizadoEm	DateTime	Última alteração.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
adicionarItem(produto, tamanho, qtd)	void	
removerItem(itemId)	void	
atualizarQuantidade(itemId, qtd)	void	
calcularTotal()	Double	
limpar()	void	
validarDisponibilidade()	List<ItemCarrinho>	Itens que ficaram indisponíveis.
aplicarCupom(codigo)	void	*(novo — UC27)*

ItemCarrinho

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
carrinhoId	Long	
produtoId	Long	
quantidade	Integer	Quantidade no carrinho.
precoUnitario	Double	Preço no momento da adição.
tamanho	String	Tamanho selecionado.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
calcularSubtotal()	Double	Preço × quantidade.
validarEstoque()	Boolean	Disponibilidade atual.

Favorito *(nova classe — UC24)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	`Long`	Identificador único.
criadoEm	`DateTime`	Data de criação.
clienteId	`Long`	
produtoId	`Long`	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
adicionar()	`void`	
remover()	`void`	

7.5. Pedidos

Pedido

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	`Long`	Identificador único.
clienteId	`Long`	*(novo)*
lojaId	`Long`	*(novo)*
itens	`List<ItemPedido>`	Composição — itens que compõem o pedido. Ciclo de vida atrelado ao Pedido (1:1..*).
enderecoEntregaId	`Long`	*(novo)*
cupomId	`Long`	Nullable. *(novo)*
status	`String`	PENDENTE → CONFIRMADO → PRONTO_PARA_COLETA → EM_ENTREGA → ENTREGUE (ou CANCELADO).
codigoColeta	`String`	*(novo — UC52/UC57)*
codigoEntrega	`String`	*(novo — UC53/UC57)*
tempoEntregaMin	`Integer`	Estimativa em minutos.
valorTotal	`Double`	Valor total cobrado.
valorFrete	`Double`	Valor do frete.
valorDesconto	`Double`	*(novo)*
motivoCancelamento	`String`	*(novo)*
canceladoPor	`String`	CLIENTE, LOJISTA, ADMIN, SISTEMA. *(novo)*
criadoEm	`DateTime`	Criação do pedido.
atualizadoEm	`DateTime`	Última atualização.

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
confirmar()	void	
cancelar(motivo)	void	
calcularTotal()	Double	
atualizarStatus(novoStatus)	void	Registra também em HistoricoStatusPedido.
podeSerCancelado()	Boolean	
gerarCodigoColeta()	String	*(novo)*
gerarCodigoEntrega()	String	*(novo)*
aplicarCupom(cupom)	void	*(novo)*

ItemPedido

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
pedidoId	Long	
quantidade	Integer	Quantidade adquirida.
precoUnitario	Double	Preço no momento da compra.
tamanho	String	Tamanho adquirido.
produtoId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
calcularSubtotal()	Double	Preço × quantidade.

HistoricoStatusPedido *(nova classe — UC30, UC58)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
statusAnterior	String	
statusNovo	String	
alteradoEm	DateTime	Base da timeline exibida ao consumidor (UC30).
origem	String	ATOR ou SISTEMA.
pedidoId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
`registrar()`	`void`	Persiste a transição de status.

7.6. Pagamento e Financeiro

Pagamento

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
`id`	`Long`	Identificador único.
`cartaoId`	`Long`	Nullable — PIX não usa. *(novo)*
`valor`	`Double`	Valor processado.
`status`	`String`	PENDENTE, APROVADO, RECUSADO, ESTORNADO.
`dataHora`	`DateTime`	Data/hora do processamento.
`transacaoId`	`String`	ID no gateway.
`gateway`	`String`	Gateway utilizado.
`pedidoId`	`Long`	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
`processar()`	`void`	
`estornar()`	`void`	
`confirmarCaptura()`	`void`	
`consultarStatus()`	`String`	

Cartao

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
`id`	`Long`	Identificador único.
`numeroMascarado`	`String`	Últimos dígitos.
`titular`	`String`	Nome impresso.
`validade`	`String`	MM/AA.
`token`	`String`	Token do gateway.
`apelidoCartao`	`String`	Apelido.
`principal`	`Boolean`	Cartão padrão.

ATRIBUTO	TIPO	DESCRIÇÃO
clienteId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
validar()	Boolean	
tokenizar()	void	
definirComoPrincipal()	void	*(novo)*

ContaBancaria

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
banco	String	Código/nome do banco.
agencia	String	Agência.
conta	String	Conta.
tipoConta	String	Corrente, poupança.
titular	String	Nome do titular.
lojistaId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
validarDados()	Boolean	Confere titularidade e formato.

TaxaComissao *(nova classe — UC63)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
percentual	Double	Percentual de comissão.
dataVigenciaInicio	DateTime	Início da vigência.
criadoEm	DateTime	Data do registro.
lojaId (Nullable)	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
aplicar(loja)	void	Define a taxa vigente para a loja (ou padrão da plataforma).
simularImpacto()	Double	Estima o efeito nos próximos repasses.

Repasse *(nova classe — UC47)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
valorBruto	Double	Valor da venda.
comissao	Double	Valor retido.
valorLiquido	Double	Valor repassado.
status	String	PREVISTO, REALIZADO.
dataPrevista	DateTime	D+2 após entrega confirmada.
dataRealizada	DateTime	Nullable.
lojistaId	Long	
pedidoId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
calcular(pedido, taxaVigente)	void	
marcarComoRealizado()	void	

7.7. Logística e Entrega

Entrega

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
localizacaoAtual	String	Descrição textual.
latAtual	Double	Latitude.
lngAtual	Double	Longitude.
dtRetirada	DateTime	Retirada na loja.
dtEntrega	DateTime	Entrega ao consumidor.
dtEstimada	DateTime	Estimativa.

ATRIBUTO	TIPO	DESCRIÇÃO
status	String	PENDENTE, EM_COLETA, EM_ENTREGA, ENTREGUE.
pedidoId	Long	
entregadorId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
confirmarColeta()	void	
confirmarEntrega()	void	
atualizarLocalizacao(lat, lng)	void	
validarGeofencing(lat, lng, raioM)	Boolean	

ProblemaEntrega

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
tipo	String	Consumidor ausente, endereço, produto danificado, etc.
descricao	String	Texto livre.
dtRelato	DateTime	Data/hora do relato.
status	String	ABERTO, EM_TRATAMENTO, RESOLVIDO, ESCALADO.
relatadoPor	String	CONSUMIDOR ou ENTREGADOR. *(novo — UC34 também relata)*
entregaId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
registrar()	void	
escalarParaAdmin()	void	

7.8. Suporte, Engajamento e Auditoria

Notificacao

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.

ATRIBUTO	TIPO	DESCRIÇÃO
titulo	String	Título.
mensagem	String	Conteúdo.
tipo	String	PEDIDO, ENTREGA, PROMOCAO, SISTEMA.
canal	String	PUSH, EMAIL, IN_APP. *(novo)*
lida	Boolean	Lida pelo usuário.
enviadaEm	DateTime	Data de envio.
usuarioId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
marcarComoLida()	void	
enviar()	void	

Avaliacao

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
nota	Integer	1 a 5.
comentario	String	Comentário opcional.
criadaEm	DateTime	Data de publicação.
statusModeracao	String	PUBLICADA, REMOVIDA. *(novo)*
tipo	String	(PRODUTO, PEDIDO, LOJA)
clienteId	Long	
lojaId	Long	
produtoId(nullable)	Long	
pedidoId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
publicar()	void	Após entrega confirmada (janela de 7 dias).
moderar()	void	Ação do administrador.

REGRA

TutorialConcluido *(nova classe — UC64)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
area	String	Área/tela do tutorial.
status	String	CONCLUIDO, DISPENSADO.
atualizadoEm	DateTime	Última atualização.
usuarioId	Long	

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
concluir()	void	
dispensar()	void	
reiniciar()	void	Rever tutorial a partir das configurações.

LogAuditoria *(nova classe — UC13, UC48, UC61, UC62)*

ATRIBUTOS

ATRIBUTO	TIPO	DESCRIÇÃO
id	Long	Identificador único.
acao	String	Ex.: ATIVAR_CONTA, ALTERAR_DADOS_BANCARIOS.
entidadeAfetada	String	Nome da entidade alterada.
entidadeId	Long	Identificador do registro alterado.
motivo	String	Nullable.
criadoEm	DateTime	Data/hora do registro.
atorId	Long	obrigatório
origemAcao	String	ADMIN, PROPRIO_USUARIO, SISTEMA

MÉTODOS

MÉTODO	RETORNO	DESCRIÇÃO
registrar()	void	

8. Relacionamentos Principais

A tabela a seguir consolida os relacionamentos entre as classes do modelo, especificando a cardinalidade e o significado semântico de cada associação.

RELACIONAMENTO	CARDINALIDADE	DESCRIÇÃO
Usuario ⇢ Cliente / Lojista / Entregador	`ISA`	Especialização por `role`.
Usuario ⇠ Notificacao	`1 : 0..*`	
Usuario ⇠ Consentimento	`1 : 0..*`	*(novo)*
Usuario ⇠ SolicitacaoExclusaoConta	`1 : 0..1`	*(novo)*
Usuario ⇠ TutorialConcluido	`1 : 0..*`	*(novo)*
Cliente ⇠ Endereco	`1 : 1..*`	
Cliente ⇠ CarrinhoCompras	`1 : 1`	
Cliente ⇠ Pedido (realiza)	`1 : 0..*`	
Cliente ⇠ Cartao	`1 : 0..*`	
Cliente ⇠ Avaliacao (faz)	`1 : 0..*`	
Cliente ⇠ Favorito	`1 : 0..*`	*(novo)*
Favorito ⇢ Produto	`N : 1`	*(novo)*
Lojista ⇠ Loja (gerencia)	`1 : 1..*`	
Loja ⇢ Endereco (localizada em)	`1 : 1`	
Lojista ⇠ ContaBancaria (possui)	`1 : 0..1`	
Loja ⇠ Produto (cataloga)	`1 : 0..*`	
Loja ⇠ Cupom (emite)	`1 : 0..*`	*(novo)*
Loja ⇠ TaxaComissao (possui)	`1 : 0..1`	*(novo — se ausente, vale a taxa padrão da plataforma)*
Lojista ⇠ Repasse (recebe)	`1 : 0..*`	*(novo)*
Loja ⇠ Pedido (recebe)	`1 : 0..*`	
Loja ⇠ Avaliacao (recebe)	`1 : 0..*`	
Produto ⇠ Estoque (controla)	`1 : 1..*`	Um registro por tamanho.
Produto ⇠ ItemCarrinho (referencia)	`1 : 0..*`	
Produto ⇠ ItemPedido (registra)	`1 : 0..*`	
Produto ⇠ Avaliacao (recebe)	`1 : 0..*`	
CarrinhoCompras ⇠ ItemCarrinho (contém)	`1 : 0..*`	

RELACIONAMENTO	CARDINALIDADE	DESCRIÇÃO
Pedido ← ItemPedido (contém)	`1 : 1..*`	
Pedido ← HistoricoStatusPedido (registra)	`1 : 1..*`	*(novo)*
Pedido ← Pagamento (possui)	`1 : 1`	
Pedido → Endereco (entregue em)	`1 : 1`	
Pedido → Cupom (usa)	`N : 0..1`	*(novo)*
Pedido ← Entrega (gera)	`1 : 0..1`	
Pedido ← Avaliacao (recebe)	`1 : 0..1`	
Pedido ← Repasse (origina)	`1 : 1`	*(novo)*
Pagamento → Cartao (pode usar)	`0..1 : 0..1`	
Entregador ← Entrega (executa)	`1 : 0..*`	
Entrega ← ProblemaEntrega (pode ter)	`1 : 0..*`	
Administrador (Usuario role=ADMIN) ← LogAuditoria	`1 : 0..*`	*(novo)*
Administrador (Usuario role=ADMIN) ← TaxaComissao	`1 : 0..*`	*(novo)*

Tabela 2 — Relacionamentos principais entre as classes do modelo.

9. Considerações Finais

Esta revisão cobre integralmente os 66 casos de uso do documento de referência. Pendências técnicas herdadas do documento original (16/06/2026) e ainda não resolvidas:

1. Padronizar o atributo de telefone/nome/dataNascimento nas três especializações de `Usuario` — considerar extrair uma classe `Pessoa` abstrata (mantido como recomendação, não bloqueia compatibilidade com os UCs).
2. Tipar `Pedido.status`, `Entrega.status`, `ProblemaEntrega.status`, `Loja.status` como enums em vez de `String`.
3. Avaliar se `solicitarDevolucao()` em `Cliente` deve virar uma classe `Devolucao` própria — hoje não há um caso de uso específico de devolução pós-entrega que exija isso.